title: Reporte Semanal de Avance – Proyecto 1

Reporte Semanal de Avance – Proyecto 1

Curso: [IC7602]

Proyecto: [1]

Semana: [8]

Fecha del reporte: [24/09/2026]

1. Integrantes del grupo

Carné	Nombre completo
2021067314	Emily Sanchez
2023101220	Andres Bonilla
2023071280	Ledvin Leiva
2022104933	Fabricio Picado
2023367224	Kevin Calvo

2. Estado de las tareas

ID	Tarea	Descripción	Responsable(s)	Estado	Avance	Problemas
T-01	Realizar la investigación inicial sobre el DNS UI	Investigar sobre como implementar el componente DNS UI	Emily Sánchez Orozco	Completada	100%	Ninguno
T-02	Realizar la investigación inicial sobre el Health Checker	Investigar sobre dos tipos de Health Check: TCP y HTTP. También, investigar como implementar la simulación de la úbicación geográfica	Andrés Bonilla Solano	En proceso	70 %	Ninguno

ID	Tarea	Descripción	Responsable(s)	Estado	Avance	Problemas
T-03	Realizar la investigación inicial sobre el DNS Interceptor	Investigar sobre como implementar el componente DNS Interceptor	Kevin Calvo	Completada	100%	Ninguno
T-04	Realizar la investigación inicial sobre el DNS API	Investigar sobre la creación de la API REST en Java, integración con Supabase/Firebase y reenvío de paquetes UDP/DNS en Base64	Fabricio Picado	Completada	100%	Ninguno
T-05	Investigación e implementación del motor de resolución DNS	Investigar e implementar la lógica base de selección de IP para registros single, multi y weight	Ledvin Leiva	Completada	100%	Ninguno
T-06	Diseño de infraestructura e integración	Definir la estructura del proyecto, red entre servicios y despliegue con Kubernetes y Helm, y adaptar la infraestructura a los componentes implementados	Ledvin Leiva	Completada	100%	Ninguno
T-07	Implementación DNS UI	Avanzar con la implementación el componente DNS UI arquitectura inicial	Emily Sánchez Orozco	En proceso	100%	Ninguno
T-08	Estructura del Health Checker	Definir la estructura que se utilizará para el Health Checker	Andrés Bonilla	Completada	100%	Ninguno
T-09	Conexión a Firebase/supabase	Definir las variables de entorno y crear un cliente de lectura hacia Firebase/supabase	Andrés Bonilla	En proceso	100%	Ninguno
T-10	Diseño e implementación inicial del DNS API	Definir la estructura base de la API REST, modelos de solicitud/respuesta y componentes para codificación/decodificación Base64	Fabricio Picado	Completada	100%	Ninguno

ID	Tarea	Descripción	Responsable(s)	Estado	Avance	Problemas
T-11	Integrar supabase y docker	Definir las tablas de la base de datos e integrar supabase y docker	Emily Sánchez y Andrés Bonilla	En proceso	80%	Ninguno
T-12	Implementar DNS-Interceptor	Realizar el desarrollo del DNS-Interceptor y sus funcionalidades	Kevin Calvo	En proceso	80%	Conectarlo al DNS-Api y realizar testing
T-13	Implementar el TCP Check	Implementar la verificación de conexión TCP en tcp_check.c	Andrés Bonilla Solano	Completada	100%	Ninguno
T-14	Implementar el HTTP Check	Implementar la verificación de conexión HTTP en http_check.c	Andrés Bonilla Solano	Completada	100%	Ninguno
T-15	Integración del DNS API con Supabase	Reemplazar datos mock por integración real con Supabase	Fabricio Picado	En proceso	60%	Ninguno
T-16	Escritura real a supabase	Escribir en la base de datos	Andrés Bonilla Solano	En proceso	10%	Ninguno
T-17	Implementar la simulación según ubicación	Hacer la ubicación simulada y realizar el cálculo del RTT según ubicación	Andrés Bonilla Solano	Completada	100%	Ninguno
T-18	Implementar selección GEO y Round Trip	Implementar la lógica de selección geográfica, cálculo Haversine, selección del Health Checker más cercano, selección por menor RTT válido y manejo de fallbacks	Ledvin Leiva	Completada	100%	Ninguno

ID	Tarea	Descripción	Responsable(s)	Estado	Avance	Problemas
T-19	Validación final de infraestructura e integración	Validar los builds Docker de los cuatro componentes, la configuración Helm, el despliegue en Kubernetes y la comunicación básica entre los servicios disponibles	Ledvin Leiva	Completada	100%	Ninguno

---

3. Commits realizados

Integrante	Fecha	Commit	Descripción	Archivos principales modificados
Emily Sánchez Orozco	10/09/2026	1696cff	scaffold inicial del modulo DNS UI	.gitignore, .env.example, package.json, vite.config.js, index.html, README.md, main.jsx, App.jsx y client.js
Andrés Bonilla Solano	10/09/2026	df5cd6a	Definición de la estructura inicial del Health Checker	config.c, config.h, db_client.c, db_client.h, http_check.c, http_check.h, location.c, location.h, main.c, tcp_check.c, tcp_check.h, utils.c, utils.h
Ledvin Leiva	10/09/2026	be2f4c4	Scaffold inicial de infraestructura con Kubernetes y Helm	.env.example, README.md, start.ps1, Chart.yaml, values.yaml, ConfigMap, Secret, Deployments y Services
Ledvin Leiva	10/09/2026	8873613	Ajustes al scaffold de Kubernetes para integrar DNS UI	dns-ui/.dockerignore, dns-ui/.env.example, dns-ui/Dockerfile, k8s/ic7602-p1/templates/configmap.yaml

Integrante	Fecha	Commit	Descripción	Archivos principales modificados
Emily Sánchez Orozco	16/09/2026	d64d072	Formulario de creación de registros DNS	records.js, supabaseClient.js, IpListEditor.jsx, RecordForm.jsx
Kevin Calvo	16/09/2026	d5de094	Scaffolding del DNS-Interceptor	dns-interceptor/*
Kevin Calvo	16/09/2026	756d732	Readme.md explicando Scaffolding del DNS-Interceptor	dns-interceptor/README.md
Emily Sánchez Orozco	17/09/2026	15a7ab0	CRUD de IP-to-Country	App.jsx, geoIp.js, records.js, GeoIpForm.jsx, GeoIpList.jsx, RecordForm.jsx, RecordList.jsx, validators.js
Emily Sánchez Orozco	17/09/2026	76155a9	Dns-ui integrado con Docker e implementación completa	global.css, App.jsx, main.jsx, Dockerfile, index.html, nginx.conf
Kevin Calvo	17/09/2026	46c19f7	Implementacion del DNS-Interceptor	dns-interceptor/*
Andrés Bonilla Solano	14/09/2026	3f150e7	Cargar configuración leyendo las variables de entorno	config.c, config.h

Integrante	Fecha	Commit	Descripción	Archivos principales modificados
Andrés Bonilla Solano	14/09/2026	fd2269a	Se agregan las variables de entorno relacionadas al Health Checker	.env.example, configmap.yaml, values.yaml
Andrés Bonilla Solano	15/09/2026	8f4cd83	Se definen la interfaz y estructuras para el cliente de base de datos	db_client.h
Andrés Bonilla Solano	16/09/2026	1b70f83	Implementación del cliente de bases de datos con datos de prueba	db_client.c
Fabricio Picado	12/09/2026	dd39273	Estructura inicial del DNS API con Spring Boot y Maven	pom.xml, DnsApiApplication.java, DnsApiController.java, application.properties y tests
Fabricio Picado	14/09/2026	be926a6	Implementación del flujo de resolución DNS mediante Base64, reenvío del paquete mediante UDP al servidor DNS externo y codificación de la respuesta	DnsApiApplication.java, controller/DnsApiController.java, service/DnsResolverService.java, application.properties
Fabricio Picado	17/09/2026	7edbe7	Dockerización del DNS API mediante una imagen Java 21 y Maven Wrapper	Dockerfile, .dockerignore

Integrante	Fecha	Commit	Descripción	Archivos principales modificados
Fabricio Picado	17/09/2026	c95bbbd	Implementación de los DTOs y modificación del endpoint /api/exists para consultar registros mediante datos mock; ajuste del formato de solicitud de /api/dns_resolver	controller/DnsApiController.java, dto/DnsRecord.java, dto/DnsRecordIp.java, dto/DnsResolverRequest.java, dto/DnsResolverResponse.java, service/DnsRecordService.java
Ledvin Leiva	17/09/2026	ead19d4	Corrección y actualización de la infraestructura actual en Kubernetes	.env.example, README.md, dns-ui/.env.example, start.ps1, values.yaml, ConfigMap, Secret y Deployments
Ledvin Leiva	17/09/2026	14a6a04	Implementación de los motores de selección GEO y Round Trip	.gitignore, dns-interceptor/README.md, api_client.rs, resolver/mod.rs, resolver/selection.rs y resolver/standard_query.rs
Andrés Bonilla Solano	18/09/2026	d6acc9b	Se agrega el TCP Check	tcp_check.c, tcp_check.h
Andrés Bonilla Solano	18/09/2026	745296f	Se agrega el HTTP Check	http_check.c, http_check.h
Andrés Bonilla Solano	20/09/2026	c9d3445	Se evalua la salud de un Target ejecutando reintentos y aplicando un algoritmo de votación por mayoría simple	health_evaluator.c, health_evaluator.h, health_checker/Makefile

Integrante	Fecha	Commit	Descripción	Archivos principales modificados
Andrés Bonilla Solano	20/09/2026	a6a5928	Se eliminan archivos obsoletos	utils.c, utils.h
Andrés Bonilla Solano	21/09/2026	f80d816	Se agrega ejecución multihilo	target_task.c, target_task.h, health_checker/Makefile
Andrés Bonilla Solano	21/09/2026	325634a	Se añaden utilidades de ubicación geográfica y la distancia de Haversine	location.c, location.h, main.c, health_checker/Makefile
Andrés Bonilla Solano	21/09/2026	dd9b3fa	Se actualizaron la cantidad de targets mock para pruebas	db_client.c
Andrés Bonilla Solano	21/09/2026	73c4367	Se solucionaron errores de compilacion y entorno para ejecutar pruebas. Además, se incluyó la documentación del Health Checker en el README.md	db_client.c, tcp_check.h, health_checker/Makefile, health_checker/README.md
Ledvin Leiva	24/09/2026	2662a0d	Validación y actualización final de la infraestructura del proyecto	README.md, dns-ui/README.md, start.ps1

Integrante	Fecha	Commit	Descripción	Archivos principales modificados
Ledvin Leiva	24/09/2026	0249b69	Ajustes finales al despliegue y manejo de configuración de infraestructura	README.md, start.ps1

---

4. Pull Requests (PRs)

Integrante	PR	Descripción	Estado	Tareas relacionadas
Emily Sánchez Orozco	1	Inicializa el proyecto DNS UI (React + Vite) con la estructura de carpetas base y configuración mínima para desarrollo local	Completado	T-07
Ledvin Leiva	2	Implementación del scaffold base de infraestructura utilizando Kubernetes y Helm, incluyendo Deployments, Services, ConfigMap, Secret, variables configurables y script de despliegue	Completado	T-06
Ledvin Leiva	3	Ajustes al scaffold de Kubernetes para permitir la integración del DNS UI dentro de la infraestructura	Completado	T-06
Andrés Bonilla Solano	4	Implementación del módulo de configuración e interfaz inicial del cliente de bases de datos	Completado	T-08 y T-09
Emily Sánchez Orozco	5	Formulario de creación de registros DNS, integración con Supabase, CRUD de registros e IP-to-Country y Dockerización del DNS UI	Completado	T-07 y T-11

Integrante	PR	Descripción	Estado	Tareas relacionadas
Kevin Calvo	6	Implementa el DNS Interceptor en Rust: parseo de paquetes DNS, clasificación Caso 1/Caso 2, cliente HTTP hacia DNS API y soporte para registros single/multi/weight	Completado	T-12
Fabricio Picado	7	Implementación inicial del DNS API con Spring Boot, resolución DNS mediante UDP hacia un servidor externo, manejo de paquetes Base64 y Dockerización con Java 21	Completado	T-10
Ledvin Leiva	8	Corrección y validación de la infraestructura actual en Kubernetes y Helm, ajustando puertos, variables de entorno, build del DNS UI, configuración del DNS Interceptor y despliegue condicional del Health Checker	Completado	T-06
Fabricio Picado	9	Incorporación de DTOs para registros y solicitudes DNS, reorganización del controlador y creación de un servicio mock para simular consultas de registros DNS	Completado	T-10
Ledvin Leiva	10	Implementación del motor de selección para registros GEO y Round Trip, incluyendo selección geográfica, cálculo Haversine, selección del Health Checker más cercano, menor RTT válido, fallbacks y pruebas unitarias	Completado	T-05
Andrés Bonilla Solano	11	Implementación de HTTP/TCP checks y decisión de healthy/unhealthy por mayoría simple. Además, se incluye la funcionalidad de que los targets se chequeen de forma simultanea y la funcionalidad del calculo de la distancia entre dos coordenadas geográficas	Completado	T-13, T-14, T-17
Ledvin Leiva	13	Validación final de infraestructura e integración: actualización de documentación, construcción de los cuatro componentes, validación de Helm, despliegue en Kubernetes, manejo seguro de configuración y verificación de conectividad entre servicios	Completado	T-06 y T-18

Kevin Calvo: Esta semana no realicé pull requests porque estuve trabajando en otros proyectos y coordinando con mis compañeros para las tareas finales.

---

5. Problemas

No se presentaron bloqueos que impidieran continuar el desarrollo durante esta semana.

Durante la validación de infraestructura se comprobó que los cuatro componentes pueden construirse mediante Docker y que DNS Interceptor, DNS API y DNS UI pueden desplegarse correctamente en Kubernetes. También se validó la resolución interna del Service `dns-api` y la conectividad HTTP entre componentes.

La integración funcional completa de los registros `geo` y `round-trip` queda condicionada a que el DNS API y el Health Checker proporcionen los datos reales requeridos. Actualmente el DNS API continúa utilizando datos mock para la consulta de registros y el Health Checker todavía utiliza targets de prueba y no persiste los resultados `healthy/unhealthy` ni las mediciones RTT en Supabase.

---

6. Resumen de participación individual

Integrante	Tareas asignadas	Commits	PRs	Resumen del trabajo realizado
Emily Sánchez Orozco	T-01 y T-07	5	2	Investigación e implementación del DNS UI, incluyendo formularios, CRUD de registros e IP-to-Country, integración con Supabase y Dockerización.
Andrés Bonilla Solano	T-13, T-14 y T-17	8	2	Implementación del TCP/HTTP Check, decisión de estado healthy/unhealthy según mayoria simple, ejecución multihilo, implementación de la simulación según ubicación y pruebas locales del componente Health Checker.

Integrante	Tareas asignadas	Commits	PRs	Resumen del trabajo realizado
Fabricio Picado	T-04, T-10 y T-15	4	2	Investigación inicial e implementación del DNS API con Spring Boot. Se implementó la estructura base de la API REST, resolución de paquetes DNS mediante UDP, manejo de paquetes en Base64, DTOs, datos mock y Dockerización del servicio. Actualmente se encuentra en desarrollo la integración del DNS API con Supabase, reemplazando los datos mock por la consulta de registros reales.
Ledvin Leiva	T-05, T-06 y T-18	6	5	Investigación e implementación del motor de resolución DNS y desarrollo de la infraestructura con Kubernetes y Helm. Se implementaron los algoritmos GEO y Round Trip, incluyendo selección geográfica, cálculo Haversine, selección del Health Checker más cercano, selección por menor RTT y fallbacks. Durante esta semana se realizó la validación final de infraestructura: se comprobaron los builds Docker de los cuatro componentes, `helm lint`, `helm template` y el despliegue en un clúster local `kind`. DNS Interceptor, DNS API y DNS UI quedaron ejecutándose correctamente y se validó la resolución interna y conectividad HTTP entre servicios. Además, se actualizó la documentación, el manejo de configuración del despliegue y se eliminaron credenciales de Supabase que estaban presentes en ejemplos versionados.
Kevin Calvo	T-03 y T-12	3	1	Investigación de RFC 1035/2929 e implementación del DNS Interceptor en Rust: parseo/serialización de paquetes DNS, clasificación Caso 1/Caso 2, cliente HTTP hacia el DNS API, resolución de registros single/multi/weight, dockerización y tests unitarios.

---

7. Plan para la siguiente semana

ID	Tarea / Actividad	Responsable(s)	Resultado esperado
T-01	Terminar con la implementación del DNS UI	Emily Sánchez Orozco	Ya con todo implementado hacer la conexión real con endpoints del DNS API
T-02	Escritura real a supabase	Andrés Bonilla Solano	Escribir en la base de datos
T-03	Pruebas unitarias del Health Checker	Andrés Bonilla Solano	No se obtengan errores al ejecutar las pruebas
T-04	Implementar la estructura base del proyecto DNS API	Fabricio Picado	La arquitectura de carpetas y las dependencias necesarias para desarrollar la API REST
T-05	Implementar los endpoints `/api/exists` y `/api/dns_resolver`	Fabricio Picado	Recibir paquetes DNS codificados en Base64, decodificarlos
T-06	Integrar GEO y Round Trip con los datos reales del sistema	Ledvin Leiva	Conectar el motor de resolución con el país o ubicación real del cliente, las ubicaciones de los Health Checkers y las mediciones RTT una vez que estos datos sean provistos por DNS API y Health Checker
T-07	Completar la integración end-to-end en Kubernetes	Ledvin Leiva	Habilitar los componentes pendientes cuando finalicen su integración con Supabase y validar el flujo completo desde DNS Interceptor hasta DNS API, Health Checker y las políticas single, multi, weight, geo y round-trip
T-08	Integrar DNS-Interceptor con DNS-Api y realizar testing	Kevin Calvo	Dejar el DNS-Interceptor funcionando conectado a DNS-API y comprobar que funciona correctamente para el Caso 1 y 2, incluyendo todas las opciones de cada caso

ID	Tarea / Actividad	Responsable(s)	Resultado esperado
T-09	Integrar el DNS API con Supabase	Fabricio Picado	Reemplazar los datos mock del DNS API por una integración real con Supabase y dejar el endpoint `/api/exists` consultando los registros reales de la base de datos
T-10	Documentación	Todos	Terminar la documentación e inlcuir todos los apartados solicitados en la especificación del proyecto

---

8. Observaciones

Como base de datos se decidió utilizar Supabase.

La infraestructura actual fue validada mediante Docker, Helm y un clúster Kubernetes local con `kind`. Los cuatro componentes construyen correctamente; DNS Interceptor, DNS API y DNS UI se encuentran preparados para desplegarse, mientras que Health Checker permanece deshabilitado en Helm hasta completar su integración real con Supabase.

La integración runtime de las políticas `geo` y `round-trip` se realizará una vez que estén disponibles los datos reales de ubicación, estado de salud y RTT requeridos.